EAST WEST UNIVERSITY

Mini Project-1

Project Title: Securing a networked system with Public Key Infrastructure

Course Name: Cybersecurity, Law and Ethics

Course Code: CSE487

Section: 01
Group: 04

Semester: Spring 2024

Submitted by:

Name	ID
Fariha Mosharof	2020-1-60-171
Tanjilul Haq	2020-1-60-214
Zarin Tasnim Hassan	2020-1-60-169
Zarin Tasnim Nuzhat	2020-1-60-211

Submitted to:

Dr. Md. Hasanul Ferdaus

Assistant Professor

Department of Computer Science & Engineering

Date of submission: 30 March 2024

Video link:
https://drive.google.com/file/d/1NHj0IdaWs2iI21bmMwbkaoa3j6pioSe4/view?usp=sharing

Introduction: In this project our task was to secure a webserver using public key infrastructure. To implement this project successfully we had to go through with many tasks and Linux command. From this project we learned why a website needs to be secured. What would happen if we used any website that is not secure. Also, we learned about Public Key Infrastructure (PKI), RSA algorithm and many more.

Materials: To complete the project we needed,

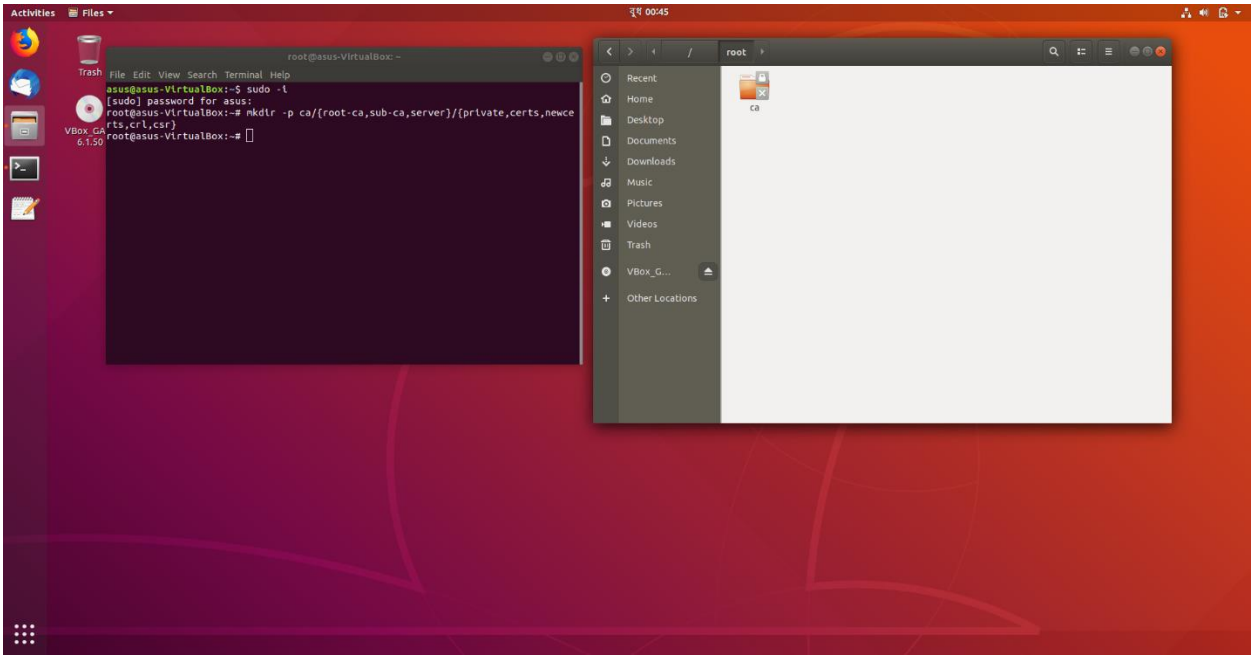
1. Virtual Machine
2. Ubuntu 18.04
3. XAMPP
4. Firefox web browser 59.0.2

Command and Instruction:

1. **CA configure:**

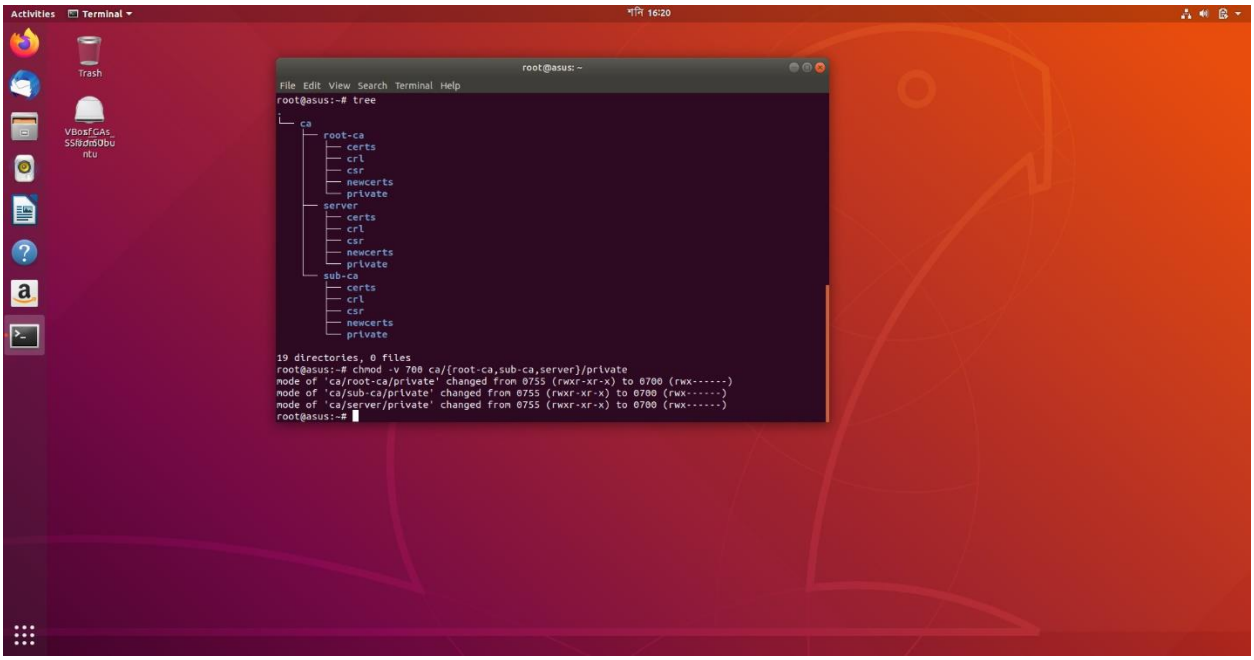
Open Root shell: sudo -i

Making directory: mkdir -p ca/{root-ca,sub-ca,server}/{private,certs,newcerts,crl,csr}



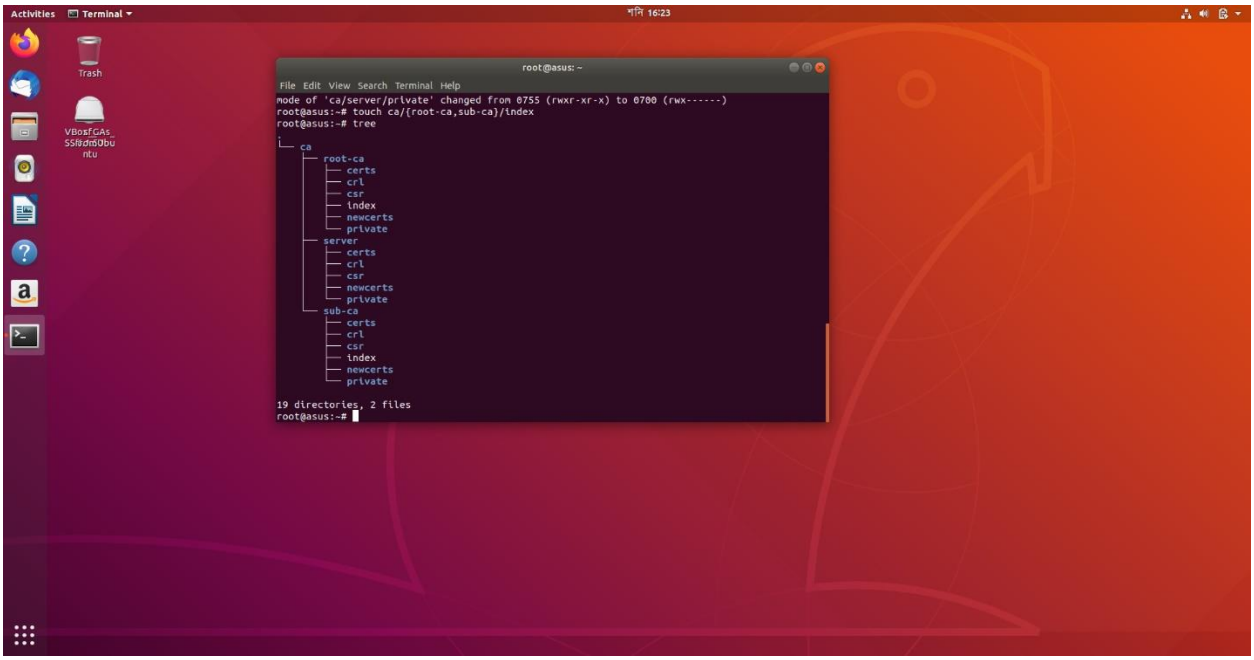
First, we need to open root shell. It provides us unrestricted access to system resources and commands. Then we created a ca folder where we gave command and created sub folder root-ca, sub-ca and server with some files.

Change permission: chmod -v 700 ca/{root-ca,sub-ca,server}/private



This allows us to set the permission of private directory files read, write and execute.

Index creation: touch ca/{root-ca,sub-ca}/index

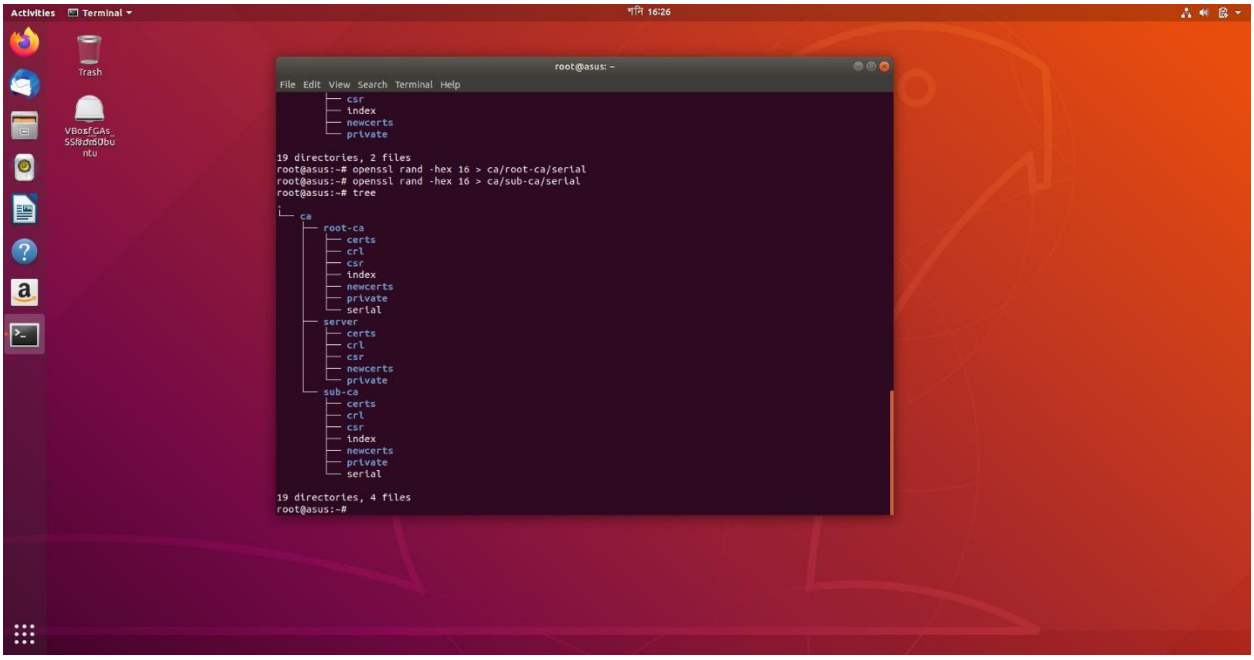


Here er created an index file for both root-ca and sub-ca.

Serial creation:

Root: openssl rand -hex 16 > ca/root-ca/serial

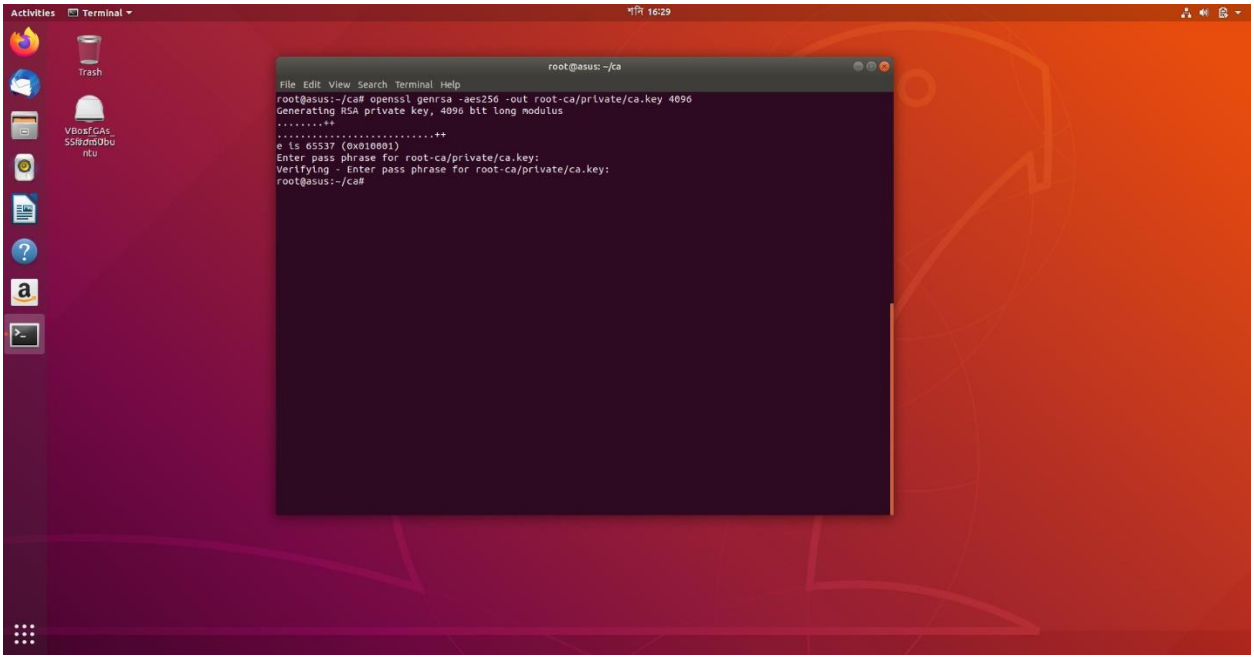
Sub ca: openssl rand -hex 16 > ca/sub-ca/serial



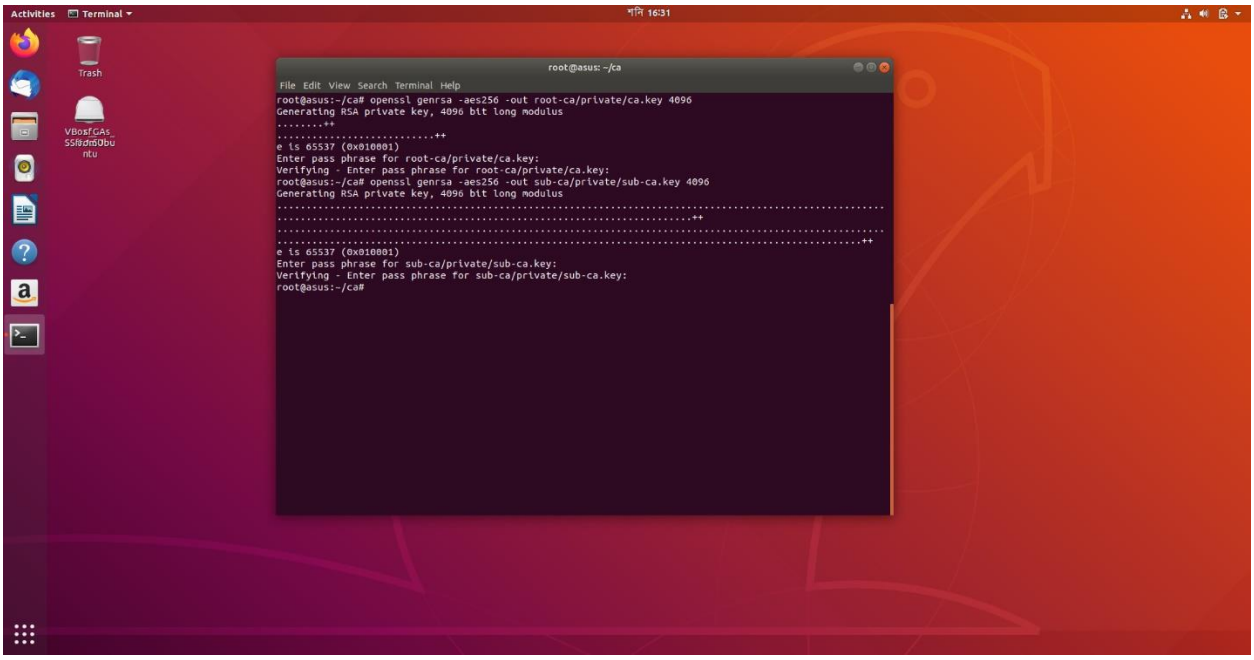
This command generates 16 bytes of random data in hexadecimal format using OpenSSL's random number generator and saves it to a file named serial in sub-ca.

2. **Private keys generation:**

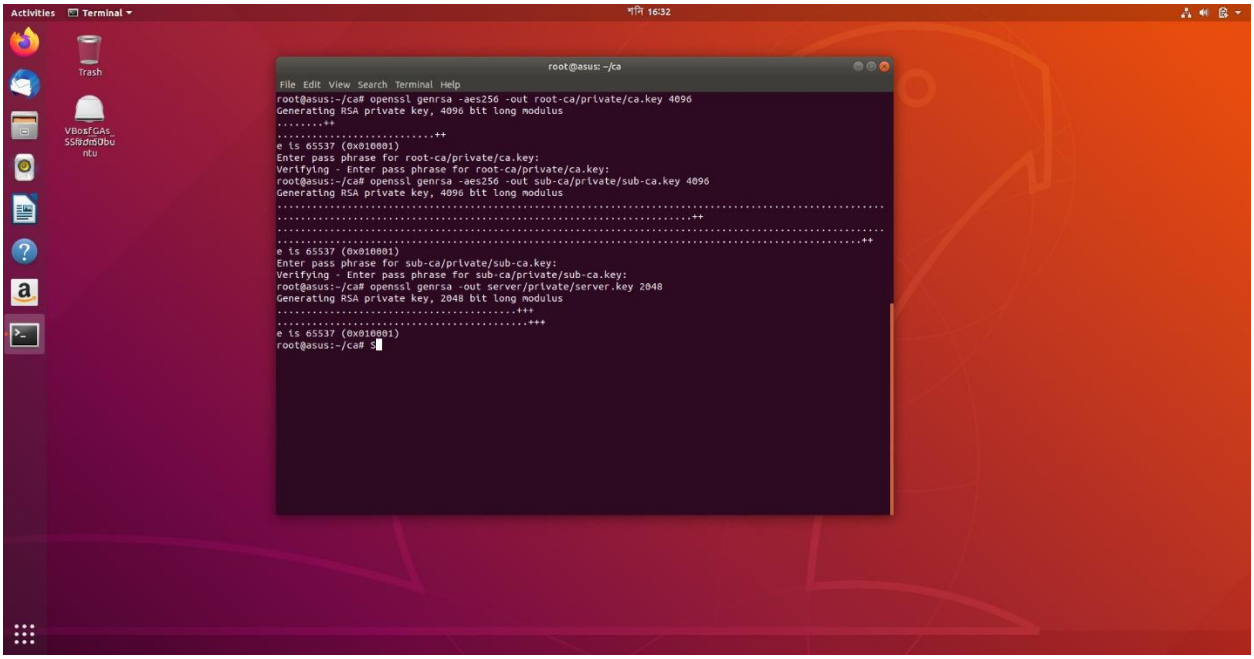
Root ca: openssl genrsa -aes256 -out root-ca/private/ca.key 4096



Sub-ca: openssl genrsa -aes256 -out sub-ca/private/sub-ca.key 4096



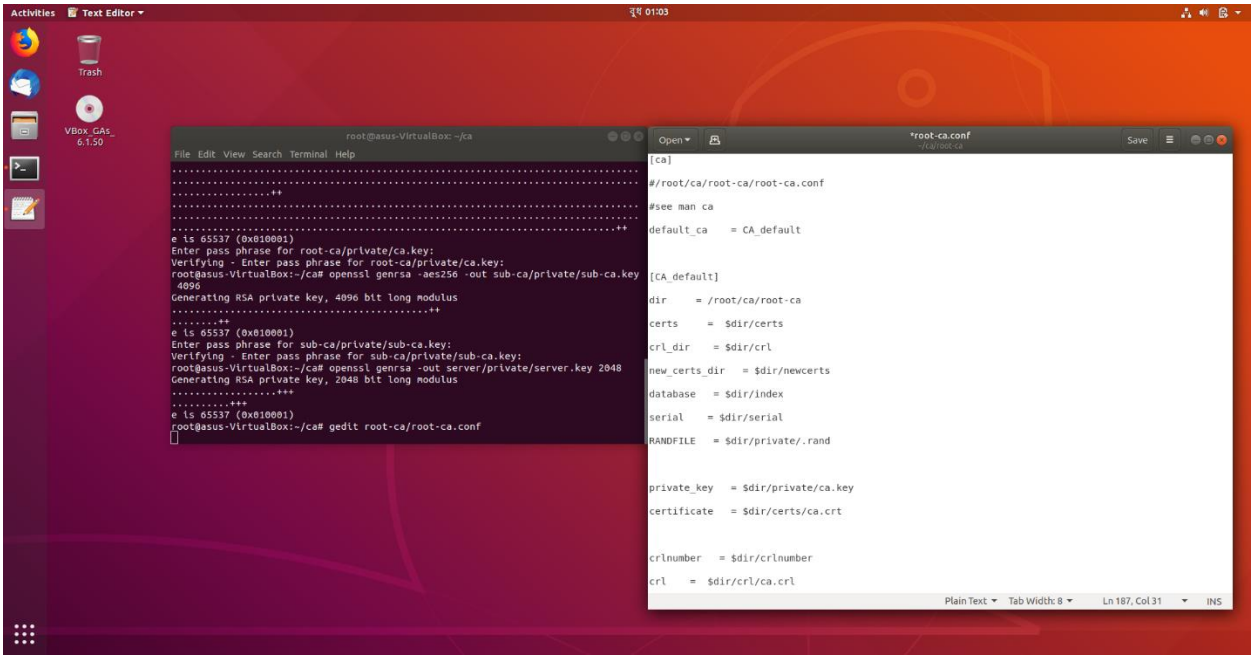
Server key: openssl genrsa -out server/private/server.key 2048



All these three commands generate an RSA private key with a length of 4096 bits, encrypts it using the AES algorithm with a 256-bit key. For generating root ca and subca we had to add password for security issue. And we use those passwords when we give permission for subca certificate and server certificate.

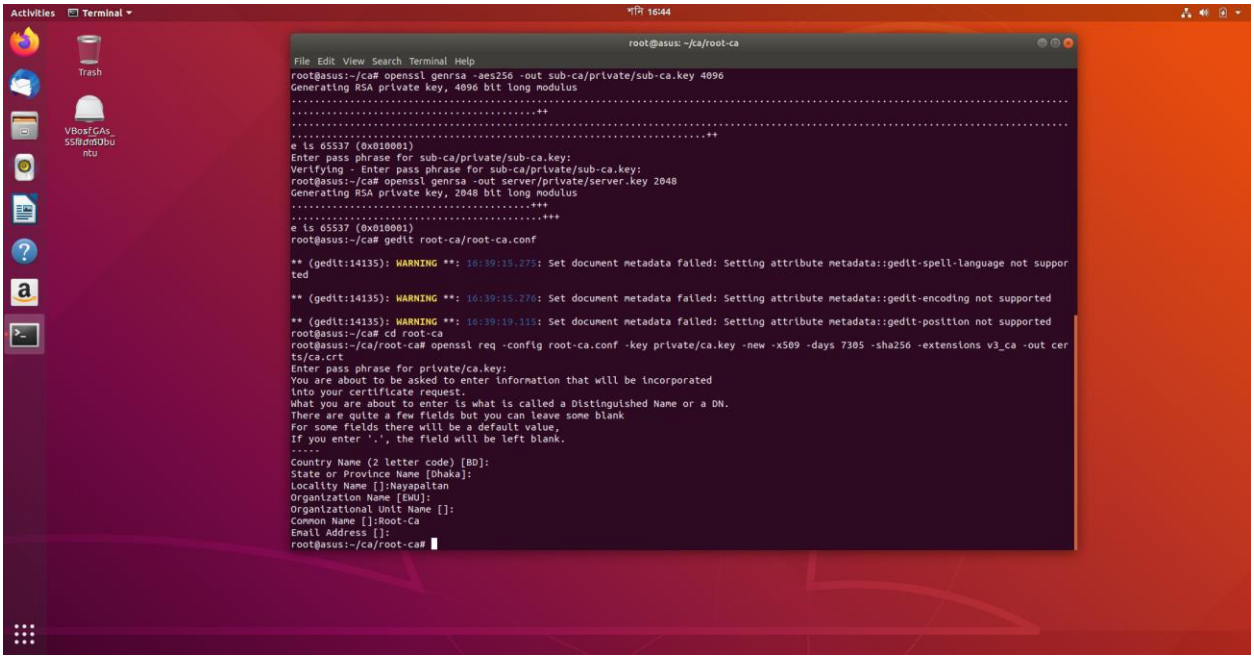
3. **Certificate generation:**

Root ca configuration: gedit root-ca/root-ca.conf



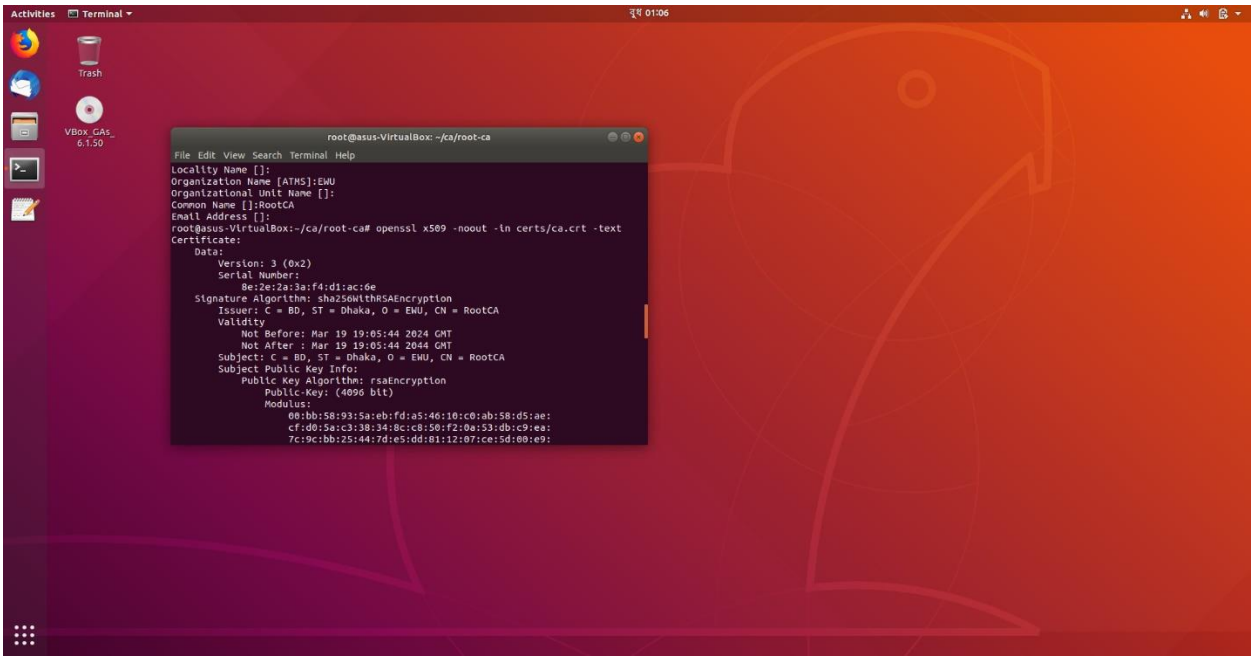
Here using **gedit** we open the root-ca.conf file in text editor and paste some code in the config file. This code are for generating the CA certificate. Here we can add the country name, common name, state name etc. then we save the file and exit.

Generate rootca: openssl req -config root-ca.conf -key private/ca.key -new -x509 -days 7305 -sha256 -extensions v3_ca -out certs/ca.crt



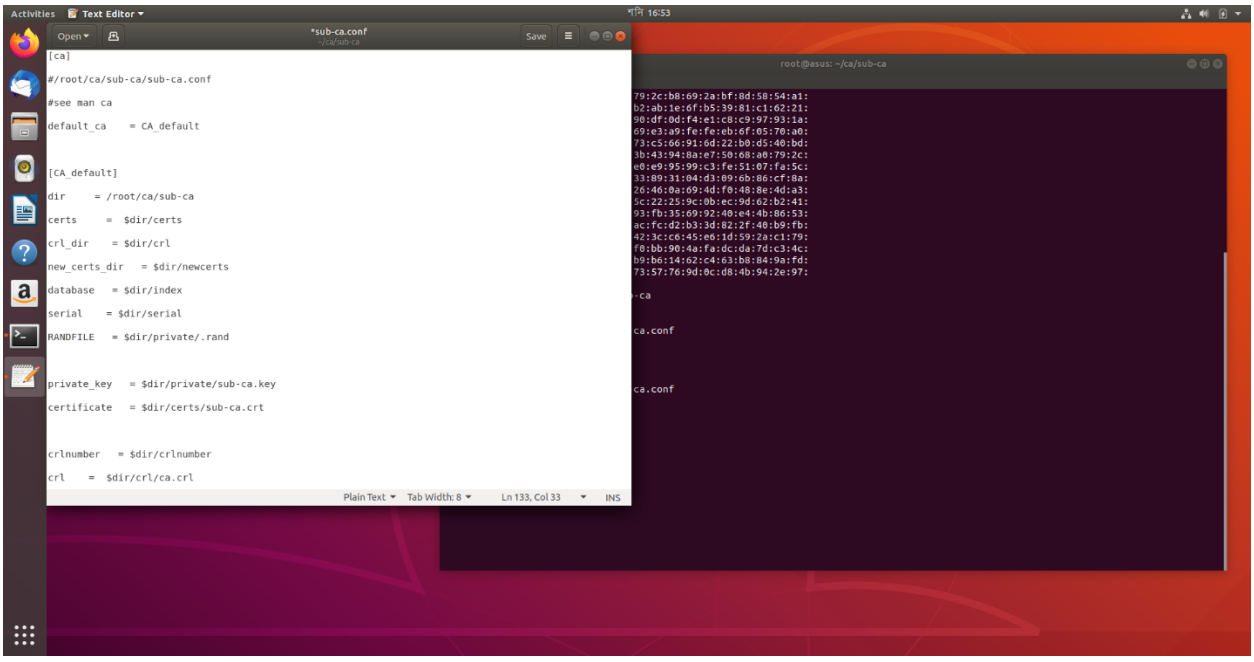
Now we generate out CA certificate. Here we could see our certificate information in root shell also if any change needed we can change the information.

Ensure rootca: openssl x509 -noout -in certs/ca.crt -text



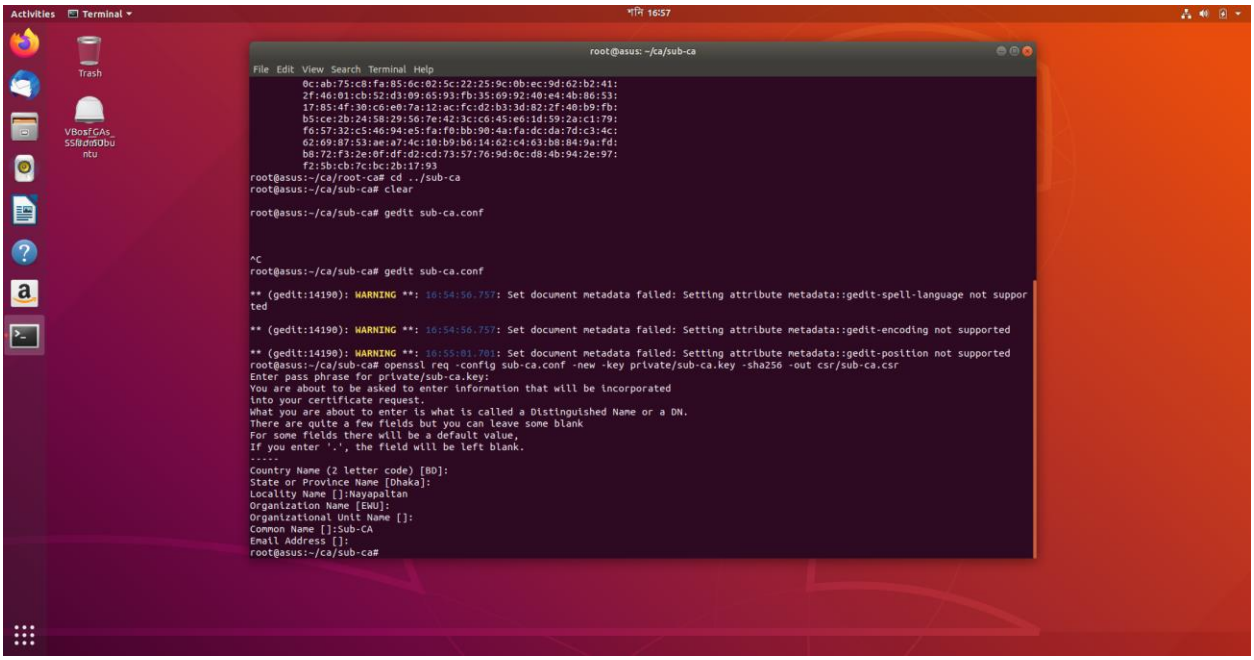
Finally, our certificate is generated. We can check our CA detail. Our detail is given.

Config subca: gedit sub-ca.conf



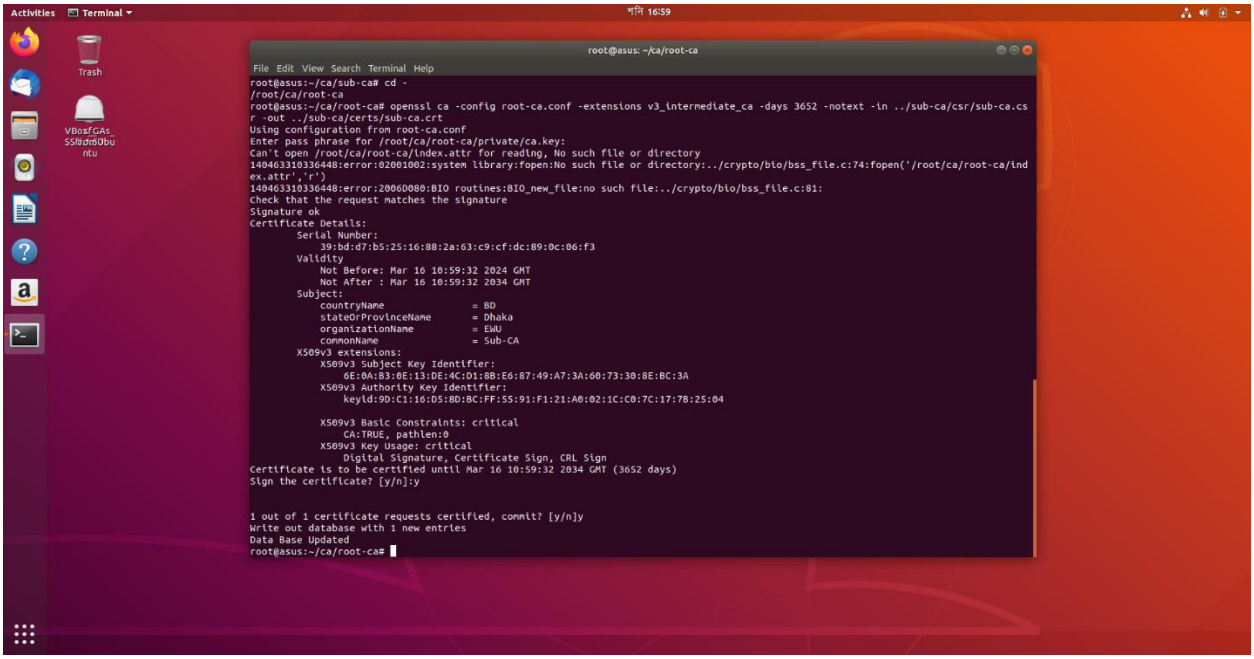
Here we use the same procedure for config file.

Requesting subCA sign to rootCA: openssl req -config sub-ca.conf -new -key private/sub-ca.key -sha256 -out csr/sub-ca.csr



Here we send a signing request from subCA to rootCA. Also, here we can check if the subCA given information is okay or not.

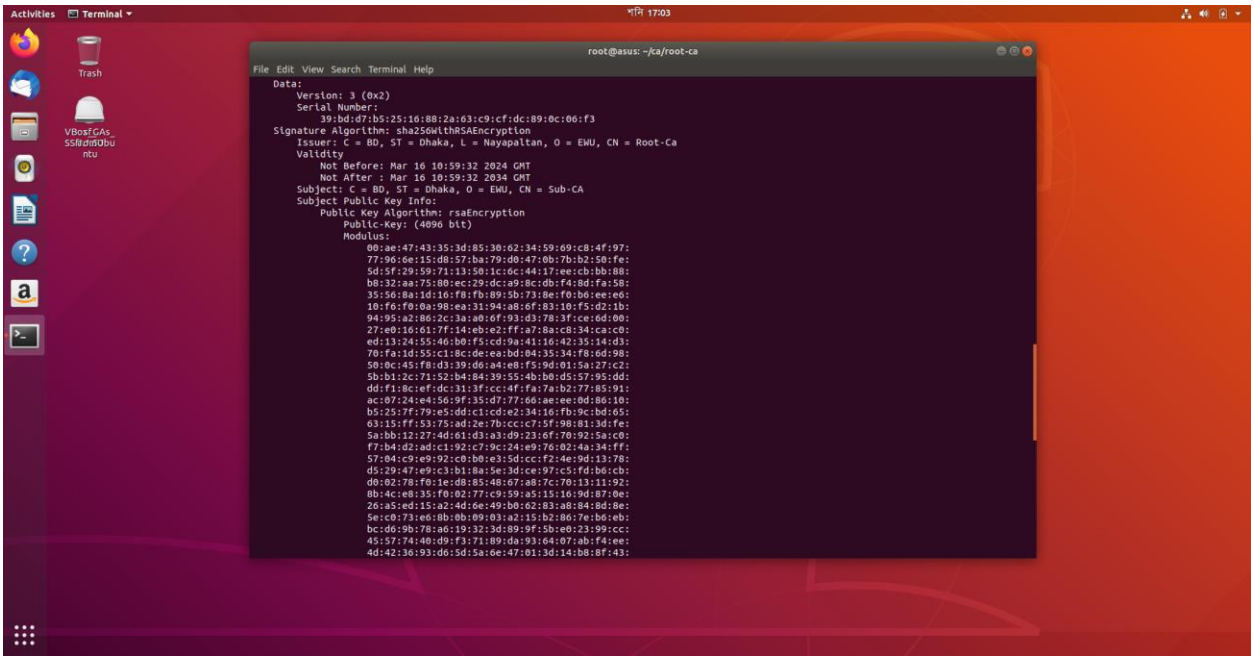
Sign by root ca: openssl ca -config root-ca.conf -extensions v3_intermediate_ca -days 3652 -notext -in ../sub-ca/csr/sub-ca.csr -out ../sub-ca/certs/sub-ca.crt



We gave the signature to subCA. For that firstly had to move to root-ca folder then we ran the given command. After executing that command it will take two permission and use the previously

saved password for rootCA. Then it will update the information in the database. Following this process, we will be able to give the permission for subCA. Now our subCA can give authorize some other server.

Detail of sub ca: openssl x509 -noout -text -in ../sub-ca/certs/sub-ca.crt

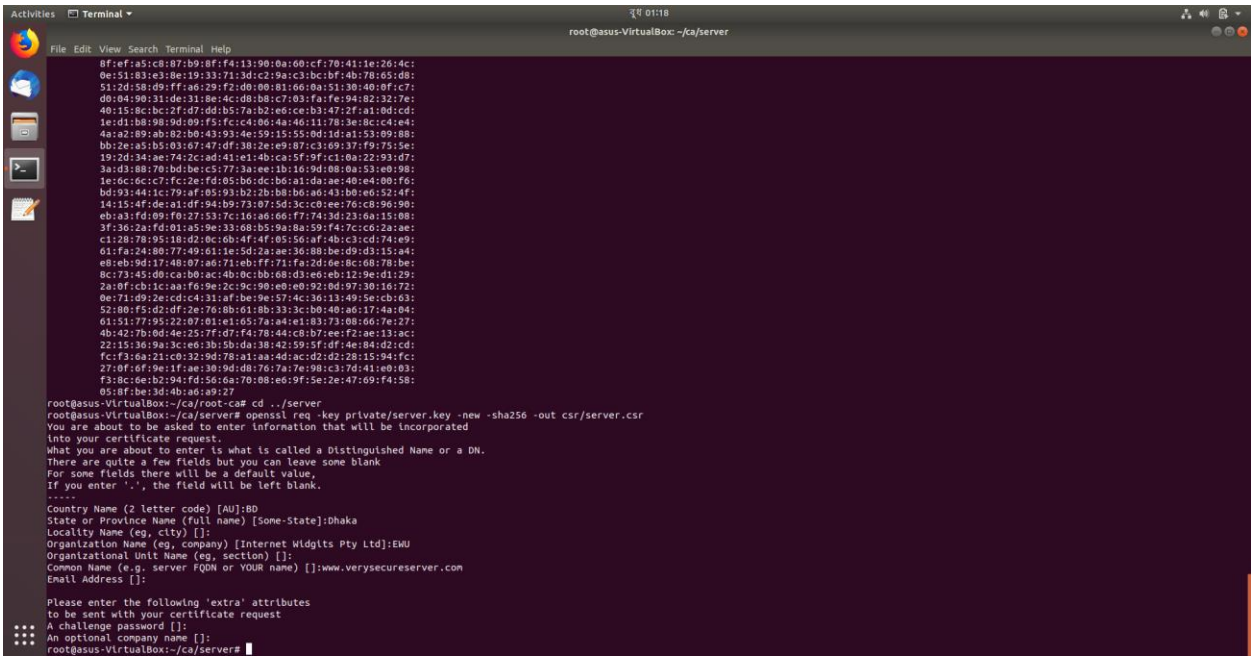


After getting the permission we check the subCA certificate if all information is okay or not.

4. **Configuring server:**

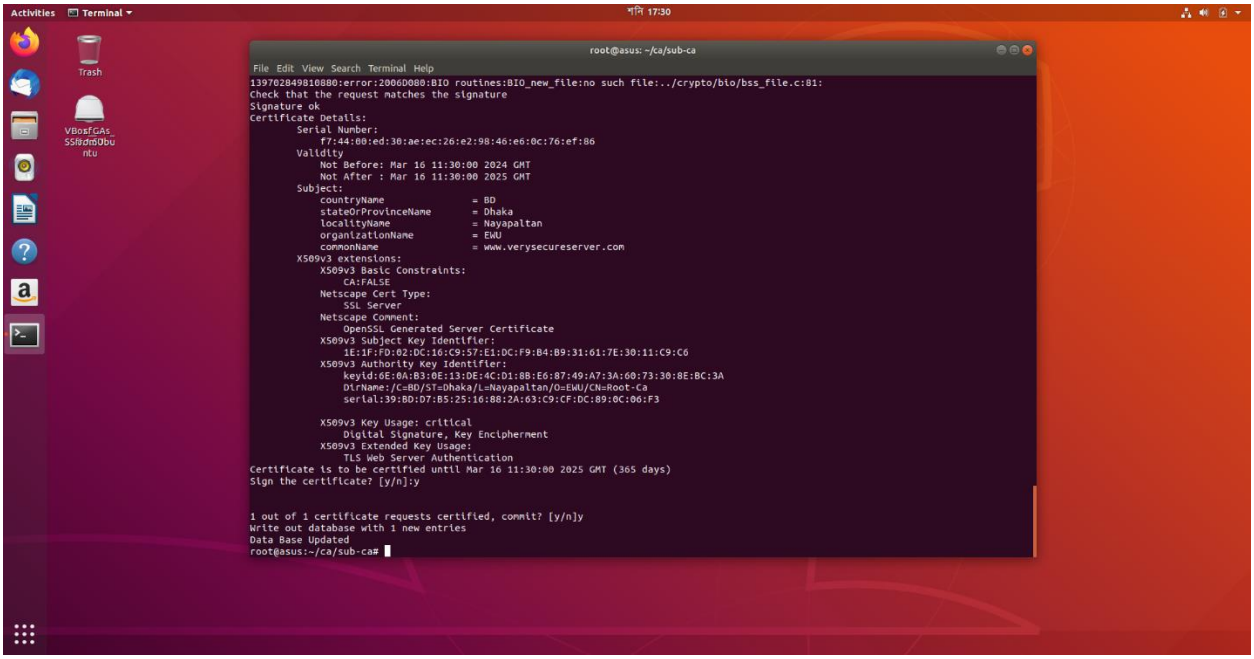
Move to server folder: cd ../server

Generate certificate request for server: openssl req -key private/server.key -new -sha256 -out csr/server.csr



After moving to the server folder we generate a certificate request for server to subCA.

Subca sign server: openssl ca -config sub-ca.conf -extensions server_cert -days 365 -notext -in ../server/csr/server.csr -out ../server/certs/server.crt

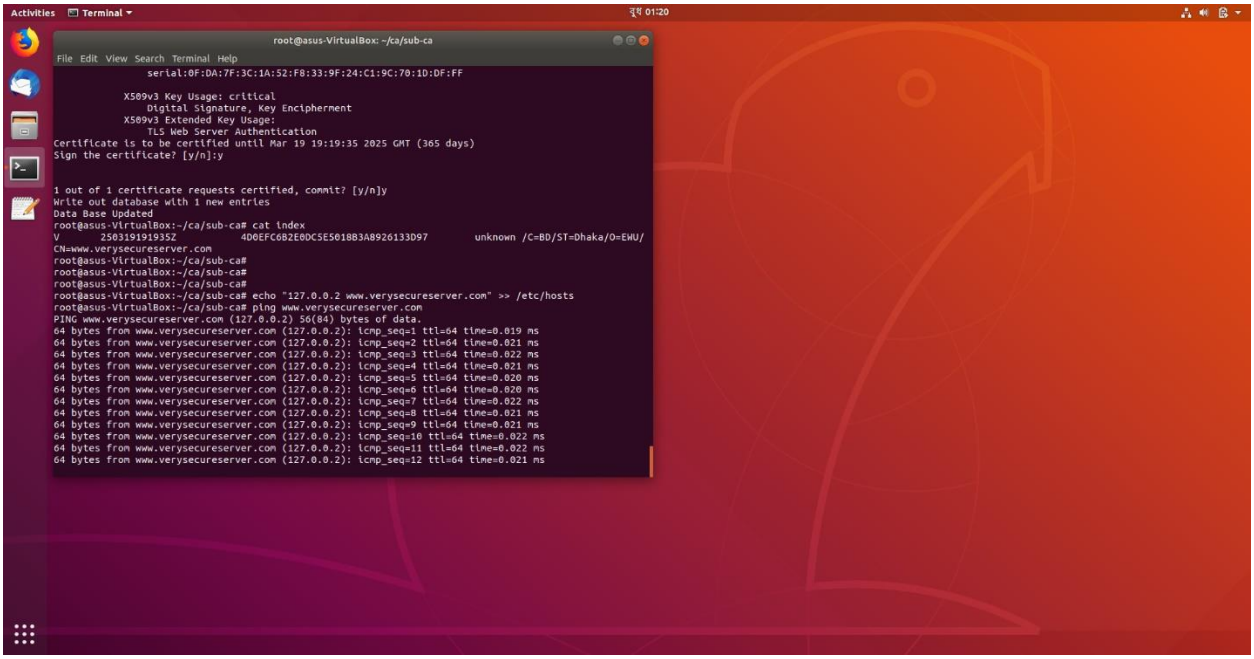


Here we ran the command for subCA to sign the server request. Here we checked all the details of server. If everything is okay, we entered “y” for signing the certificate then again entered “y” to commit. After doing this it will be updated in the database.

Check detail of server: cat index

Pinging with server: echo "127.0.0.2 www.verysecureserver.com" >> /etc/hosts

ping www.verysecureserver.com



After getting the sign we can check if our work has been done. Then we tried to ping with our website. Here we can see that the ping was successful because the packets were transferring.

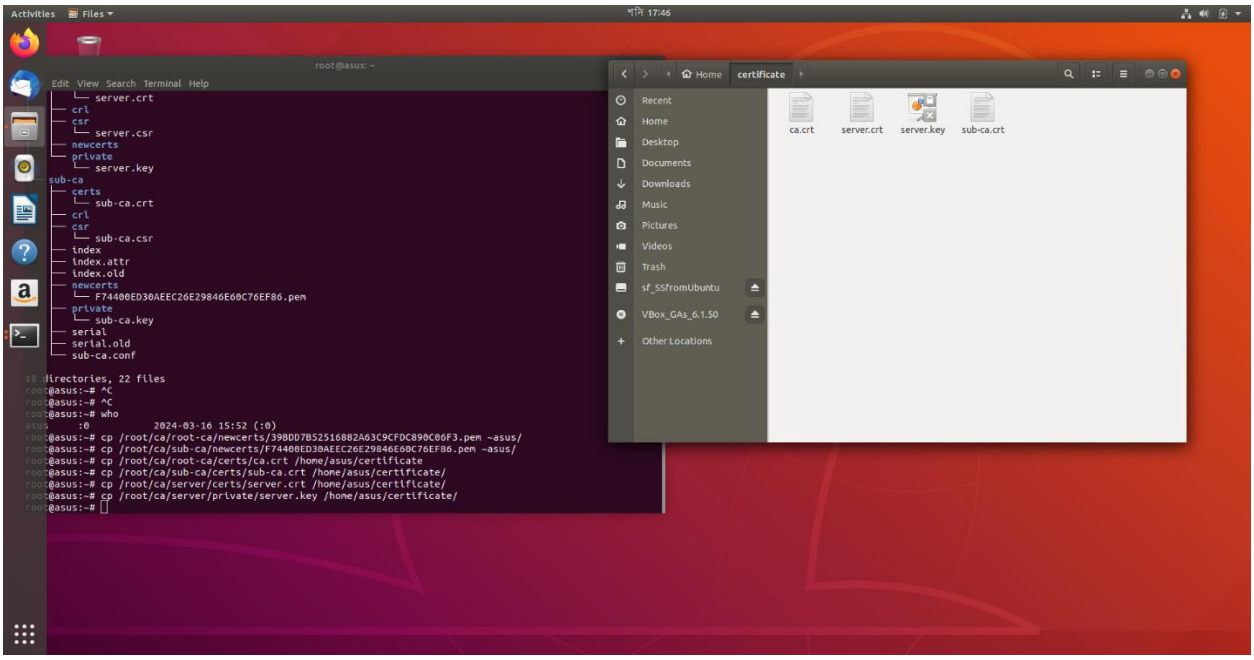
Coping certificate:

cp /root/ca/root-ca/certs/ca.crt /home/asus/certificate

cp /root/ca/sub-ca/certs/sub-ca.crt /home/asus/certificate/

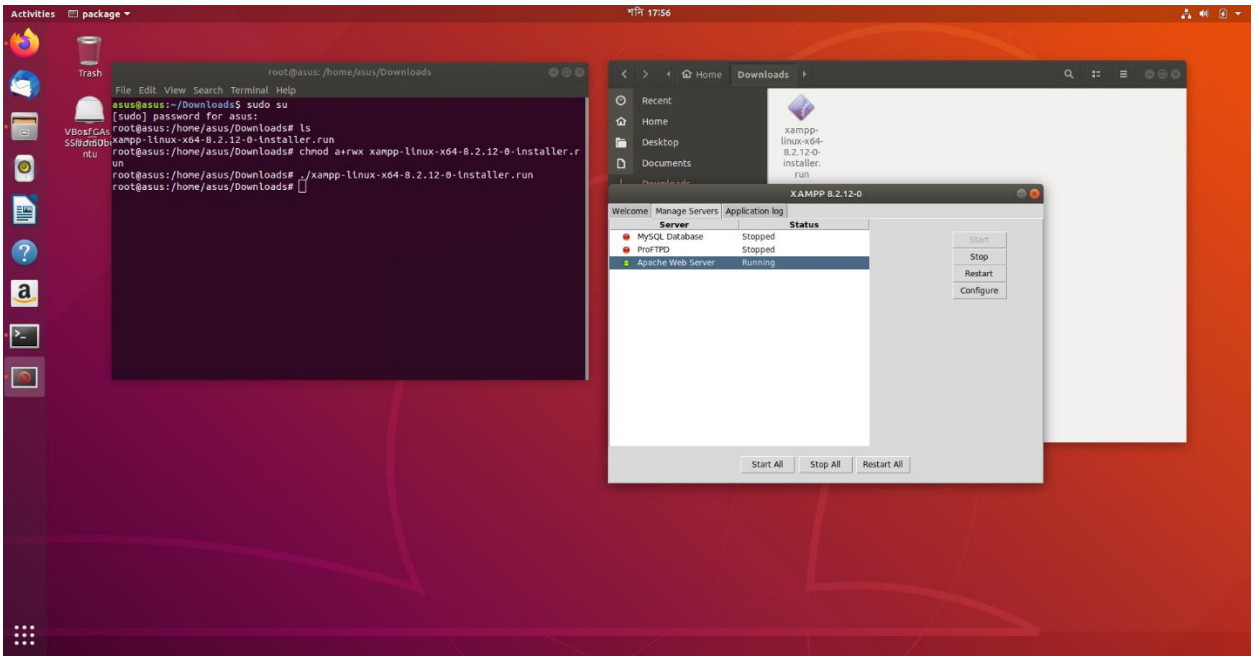
cp /root/ca/server/certs/server.crt /home/asus/certificate/

cp /root/ca/server/private/server.key /home/asus/certificate/



Now we created a folder named “certificates” then copied ca, subca, server certificate and server key file to work more easily. Otherwise, we would needed to go different different folder for desired files.

XAMPP installation:

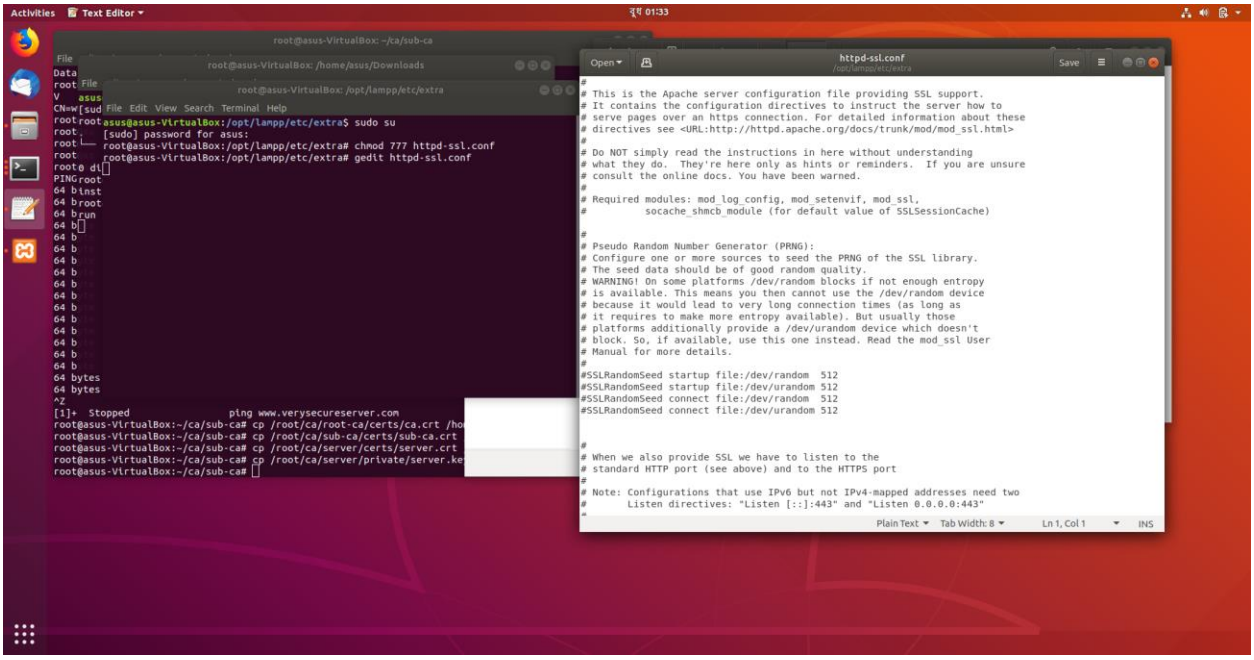


Then we installed xampp and started the apache sever.

Going to SSL config folder: cd /opt/lampp/etc/extra

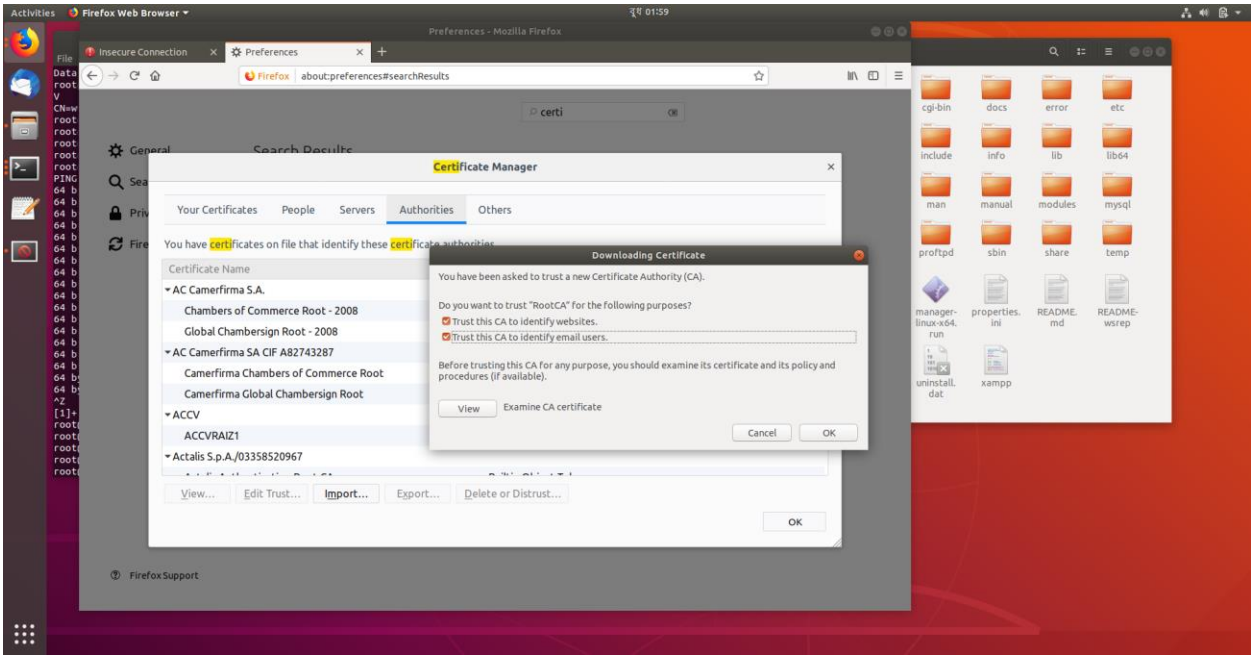
Changing permission for SSL: chmod 777 httpd-ssl.conf

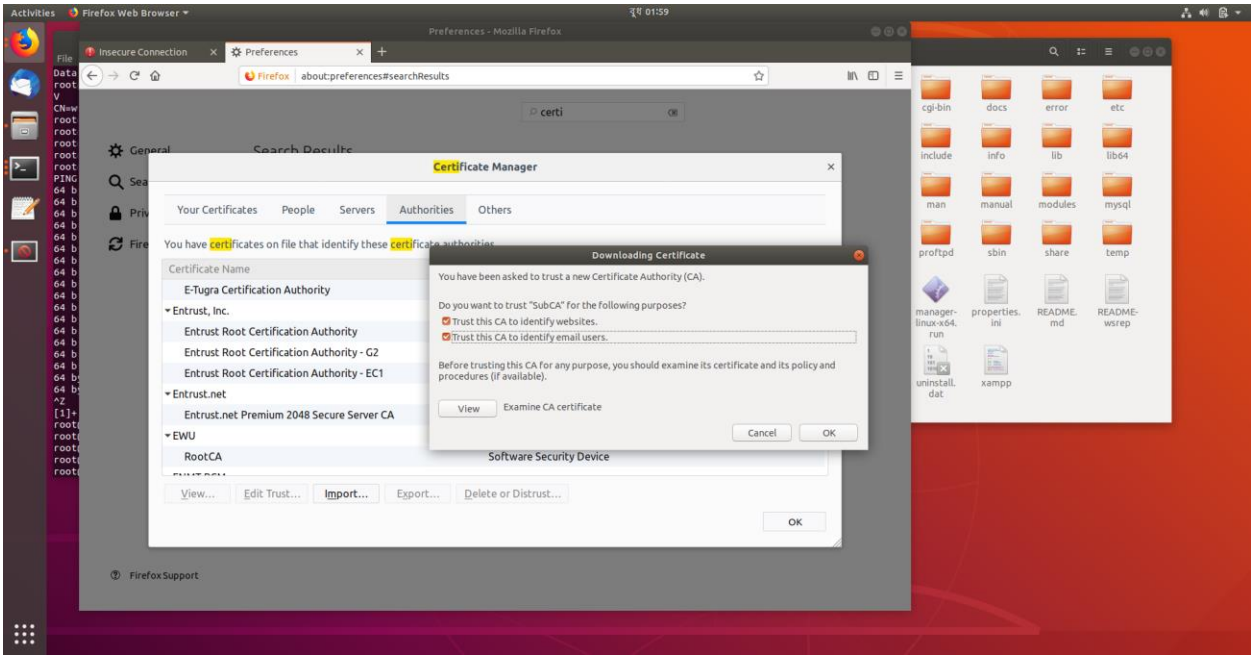
Editing SSL: gedit httpd-ssl.conf



SSL config file is by default private file and we cannot write or execute. For that we used chmod 777 to get the permission to write and execute. Then by using “gedit” we opened the file in texteditor. In texteditor we change three lines. In line number 106, we gave SSLCertificateFile "/home/asus/certificate/server.crt". this line shows the path for our server certificate path. In line 116 we changed this SSLCertificateKeyFile "/home/asus/certificate/server.key". it shows the server key path. And lastly, we gave our certificate path by using SSLCACertificatePath "/home/asus/certificate" in line 136.

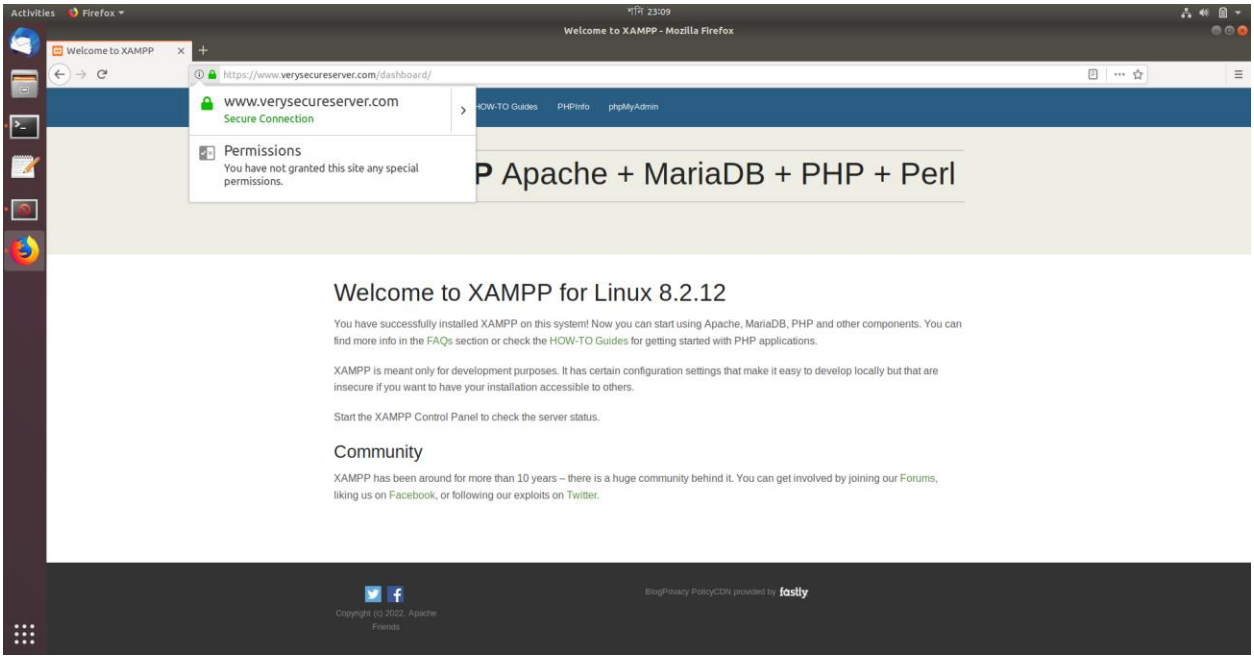
5. Uploading certificates:

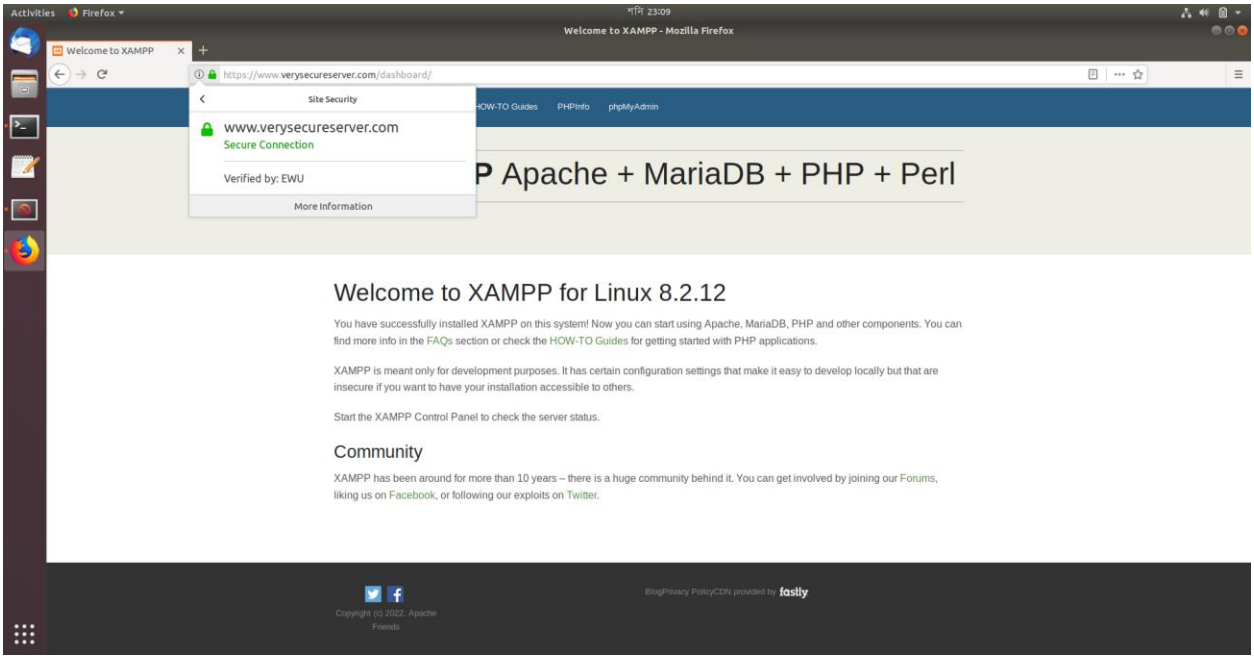




To upload our certificate, we opened the web browser. For us it was Firefox. Then went to preferences searched for certificates. Then we had a window like the above image. We went to import and first we selected CA certificated then again selected subCA certificate.

Final output:





After completing all the processes. We opened the web browser. Then we used https:// before or website link. We used https://www.verysecureserver.com. Then we had a lock icon shown in the above. It says it is secured. Then if we go to detail it shows the organization name we used.

Conclusion: After completing the project, we learned how we can secure our web site by using some steps. Also while doing the work, we faced some issue like the versions for ubuntu and Firefox. If we do not use proper version our work will not be done properly. Securing the website was our main goal which we completed successfully and showed the steps we needed to do.